Manual de Programación del Lenguaje K++

De cero a tus primeros programas que compilan a KolibriOS

Versión del manual: 1.0 — noviembre de 2026 **Autor del lenguaje y del compilador:** Esequiel Pena **Lenguaje documentado:** K++ con Stage 0 completado (8 de noviembre de 2026) y Stage 1 en desarrollo. **Base de verificación:** la suite oficial del proyecto (71 archivos versionados: 60 positivos, 11 negativos, 0 fallos) y las pruebas de campo ejecutadas en KolibriOS real. Cada afirmación de este manual corresponde a un ejercicio que compila y corre, o está marcada como limitación.

Índice

1. Introducción: ¿qué es K++?
2. Tu primer programa
3. Estructura de un programa K++
4. Tipos de datos
5. Variables y asignación
6. Operadores
7. Salida por pantalla: print y print_int
8. Control de flujo: if / else y while
9. Funciones
10. Arreglos
11. Estructuras: struct
12. Enumeraciones: enum
13. Entrada y salida de archivos
14. Comentarios
15. Errores comunes y mensajes del compilador
16. Compilar y ejecutar tu programa
17. Bajo el capó: la ABI y el ejecutable
18. Filosofía de diseño
19. Limitaciones conocidas (lo que K++ aún no hace)
20. Referencia de ejercicios verificados L01–L17

1. Introducción: ¿qué es K++?

K++ (archivos `.kpp`) es un lenguaje de programación creado desde cero, junto con su propio compilador: lexer, parser, analizador semántico, representación intermedia (IR), backend x86 y generador de ejecutables propios. No es una envoltura sobre otro compilador.

Su objetivo es ambicioso y poco común:

- Compilar programas escritos en un subconjunto de C++ a código máquina x86 de 32 bits.
- Generar ejecutables nativos de KolibriOS (archivos `.bin`), no ELF de Linux.
- Crecer por etapas hasta lograr el self-hosting: que el compilador de K++ esté escrito en K++ y pueda compilarse a sí mismo.

Estado actual. Stage 0 está **completado**: el compilador escrito en C++ compila el lenguaje completo descrito en este manual y cada pieza está sellada con una prueba de campo en KolibriOS. Stage 1 — reescribir el compilador en el propio K++ — está en marcha; su primera pieza (el lexer) se construye por rebanadas verificadas.

Una frase resume el carácter del lenguaje: **se escribe en alto nivel y se ejecuta en ensamblador**. La fuente usa funciones, `if`, `while`, structs y arreglos; la salida es un binario plano x86, sin intérprete, sin máquina virtual y sin runtime escondido entre tu programa y el procesador.

2. Tu primer programa

El clásico punto de partida. Guárdalo como `hola.kpp`:

```
int main()
{
    print("Hola K++");
    return 0;
}
```

Y compílalo:

```
./bin/kpp hola.kpp
```

Si todo va bien, el compilador muestra su cabecera, el árbol sintáctico (AST) del programa, el IR, el código máquina generado y escribe el ejecutable en la carpeta `Output/`. `print("...")` escribe texto en pantalla y `return 0;` indica que el programa terminó correctamente.

3. Estructura de un programa K++

Un programa K++ se compone de declaraciones de nivel superior, que solo pueden ser de tres tipos:

- declaraciones `struct`,
- declaraciones `enum`,
- funciones que devuelven `int`.

No existen variables globales. Todo dato vive dentro de una función. Todo programa necesita una función `main`: es el punto de entrada por donde empieza la ejecución.

```
struct Persona
{
    int edad;
    string nombre;
};

enum Color
{
    ROJO,
    VERDE,
    AZUL
};

int main()
{
    print("Estructura completa");
    return 0;
}
```

Nota: *las llaves `{ }` delimitan los bloques de código y cada instrucción termina con punto y coma `;`, como en C++.*

El compilador lee el programa **entero** antes de analizarlo, así que el orden de las funciones no importa: puedes llamar a una función definida más abajo en el archivo (capítulo 9.2). No hay declaraciones previas ni archivos de cabecera: de eso trata el capítulo 18.

4. Tipos de datos

K++ trabaja con dos tipos de datos:

Tipo	Descripción	Ejemplo
int	Número entero de 32 bits	int edad = 25;
string	Cadena de texto (literal entre comillas dobles)	string nombre = "Ana";

Importante: *no des por hecho que existen* `void`, `bool`, `float`, `double`, `char` *ni variantes como* `long` *o* `unsigned int`*. El lenguaje operativo de hoy trabaja con* `int` *y* `string`*.*

5. Variables y asignación

Las variables se declaran con su tipo y pueden inicializarse en la misma línea:

```
int main()
{
    int a = 10;
    int b = 20;
    int c = a + b;

    print("L02 VARIABLES");
    print_int(a);
    print_int(b);
    print_int(c);

    return 0;
}
```

También puedes asignar un nuevo valor a una variable ya declarada (sin repetir el tipo):

```
int contador = 3;
contador = contador - 1;
```

Reglas: *no puedes usar una variable antes de declararla, ni declarar dos veces el mismo nombre en el mismo ámbito. El analizador semántico lo detecta y el compilador lo rechaza. Una variable declarada dentro de un bloque puede ocultar (shadow) a otra exterior del mismo nombre: la interior tiene su propio espacio y la exterior no se toca.*

K++ **no tiene** los operadores `++` ni `--`. Para incrementar o decrementar se escribe la operación completa, como en BASIC clásico:

```
i = i + 1;
contador = contador - 1;
```

6. Operadores

6.1. Aritméticos

Operador	Significado	Estado
+	Suma	✔ Funciona
-	Resta	✔ Funciona
*	Multiplicación	✔ Funciona (el backend emite imul)
/	División	✔ Funciona
%	Módulo (resto)	✔ Funciona

```
int a = 20;
int b = 6;
print_int(a + b);    // 26
print_int(a - b);    // 14
print_int(a * b);    // 120
print_int(a / b);    // 3
print_int(a % b);    // 2
```

6.2. Relacionales (comparación)

Operador	Significado
==	Igual a
!=	Distinto de
<	Menor que
>	Mayor que
<=	Menor o igual
>=	Mayor o igual

6.3. Lógicos

Operador	Significado
&&	Y lógico
||	O lógico
!	Negación

```
int a = 10;
int b = 5;

if (a == 10 && b != 0)
{
    a = a + 1;
}

if (a > b || a <= 0)
{
    a = a + 1;
}

if (!(a == b))
{
    a = a + 1;
}
```

Ojo: *`&&` y `||` **no cortocircuitan**: los dos lados de la expresión se evalúan siempre. No uses el lado derecho como guarda del izquierdo (por ejemplo, no protejas un acceso a un arreglo con la condición*

que va a su izquierda en el mismo `&&`); anida dos `if` en su lugar.

7. Salida por pantalla: print y print_int

K++ ofrece dos funciones integradas de salida (implementadas en su runtime para KolibriOS):

Función	Uso	Ejemplo
print	Escribe un literal de texto o una variable string	print("Hola K++");
print_int	Escribe un valor entero (variable o expresión)	print_int(edad);

```
int main()
{
    int edad = 25;
    string nombre = "Ana";

    print("L01 TIPOS");
    print_int(edad);
    print(nombre);

    return 0;
}
```

Detalle: *`print_int` acepta expresiones, como `print_int(a + b)`. Y ten presente que imprime el entero* ***sin signo****: el valor interno 1 se muestra como `4294967295`. Es el comportamiento verificado del runtime, no un error de tu programa.*

8. Control de flujo: if / else y while

8.1. Condicionales

```
int clasificar(int x)
{
    if (x > 20)
    {
        return 3;
    }
    else if (x > 10)
    {
        return 2;
    }
    else
    {
        return 1;
    }
}

int main()
{
    print("CASO 25");
    print_int(clasificar(25));    // 3

    print("CASO 15");
    print_int(clasificar(15));    // 2

    print("CASO 5");
    print_int(clasificar(5));     // 1

    return 0;
}
```

La condición va entre paréntesis y el bloque entre llaves. Puedes encadenar `else if` tantas veces como necesites y cerrar con un `else` final.

8.2. Bucles while

```
int main()
{
    int contador = 3;

    while (contador > 0)
    {
        print_int(contador);
        contador = contador - 1;
    }

    return 0;
}
```

El bloque se repite mientras la condición sea verdadera. Asegúrate de que algo dentro del bucle modifique la condición, o el programa no terminará nunca.

9. Funciones

Las funciones agrupan código reutilizable. Se definen con tipo de retorno `int`, nombre, parámetros entre paréntesis y cuerpo entre llaves:

```
int sumar(int a, int b)
{
    return a + b;
}

int obtener_valor()
{
    return 42;
}

int main()
{
    int resultado = sumar(10, 20);
    print_int(resultado);         // 30

    int otro = obtener_valor();
    print_int(otro);              // 42

    return 0;
}
```

- **Parámetros:** se declaran con tipo y nombre (`int a, int b`). El compilador valida que la llamada pase la cantidad y los tipos correctos; equivocarse en el número de argumentos es un error de compilación (la suite lo comprueba con pruebas negativas).
- **Argumentos:** pueden ser constantes, variables o expresiones.
- **Retorno:** `return` devuelve un valor entero y termina la función.
- **Llamadas:** una llamada es una *expresión*, no una instrucción suelta. Su resultado se guarda en una variable o se usa dentro de otra expresión: `int a = f();`, `print_int(f());`, `return f();`. Escribir `f();` sola en una línea **no** es válido en K++.

9.1. Expresiones como argumentos

Los argumentos no tienen que ser solo constantes o variables: también pueden ser expresiones, que se evalúan antes de la llamada:

```
int suma(int a, int b)
{
    return a + b;
}

int main()
{
    int x = 10;
    int y = 20;

    print_int(suma(x + 5, y));       // 35
    print_int(suma(x + 5, y * 2));   // 55

    return 0;
}
```

9.2. Llamar funciones definidas después

K++ permite llamar a una función que está definida más abajo en el archivo. No necesitas declarar prototipos: el compilador lee el programa entero y resuelve las referencias hacia adelante:

```
int main()
{
    int x = 0;
    x = suma();
    print_int(x);    // 20
    return 0;
}

int suma()
{
    return 20;
}
```

9.3. Retornar directamente una llamada

Puedes devolver el resultado de una llamada sin pasar por una variable intermedia:

```
int suma()
{
    return 10;
}

int main()
{
    return suma();    // main retorna 10
}
```

Prueba de fuego — la regla del `return` que imprime: *el compilador imprime automáticamente en pantalla el valor de todo `return` cuya expresión **no** sea una constante literal. Así, `return x;`, `return a + b;` o `return suma();` muestran su valor al ejecutar el programa, mientras que `return 0;` o `return 42;` no imprimen nada. Es una ayuda intencional para validar visualmente los programas en KolibriOS, no un error del compilador.*

9.4. Recursión y llamadas anidadas

Una función puede llamarse a sí misma. La recursión está verificada en KolibriOS (ejercicio L15): el factorial recursivo imprime 2, 6, 24, 120 para 2, 3, 4 y 5. Cada llamada tiene su propio marco de pila, así que parámetros y variables locales son independientes por invocación.

También funcionan las llamadas anidadas y compuestas (ejercicio L16): el resultado de una llamada puede ser argumento de otra, guardando los intermedios en variables cuando la expresión se complica.

9.5. Cuántos argumentos: la convención de llamada

K++ acepta funciones con cualquier número de parámetros. Por dentro, los dos primeros argumentos viajan en registros (EAX y EBX) y del tercero en adelante viajan por la pila; el resultado vuelve en EAX. El capítulo 17 lo explica completo. Para quien escribe programas, la consecuencia práctica es simple: **no hay límite práctico de parámetros** — está verificado con funciones de tres, cuatro y más argumentos (ejercicios `sonda_ciclo4` y `function_parameters`).

10. Arreglos

Un arreglo guarda varias celdas `int` bajo un solo nombre. Se declara con su tamaño entre corchetes y cada celda se direcciona con un índice que empieza en 0:

```
int main()
{
    int valores[4];
    valores[0] = 42;
    valores[1] = valores[0] + 8;
    print_int(valores[1]);    // 50
    return 0;
}
```

- El índice puede ser una constante o una expresión (`valores[i]`, `valores[i + 1]`), así que los arreglos se recorren con `while` de forma natural.
- Un índice constante fuera de los límites es **error semántico** en tiempo de compilación: el compilador lo rechaza con el mensaje «índice fuera de límites para el array». Es una de las pruebas negativas de la suite.

10.1. Arreglos compartidos entre funciones

Un arreglo se comparte **por nombre**: si dos funciones declaran un arreglo con el mismo nombre, están viendo **las mismas celdas**. No hace falta pasarlo como parámetro (K++ no pasa arreglos como argumentos):

```
int escribe()
{
    int t[4];
    t[0] = 72;
    return 0;
}

int lee()
{
    int t[4];
    print_int(t[0]);    // 72: es la misma tabla que tocó escribe()
    return 0;
}

int main()
{
    int a = escribe();
    int b = lee();
    return 0;
}
```

Este es el ejercicio `sonda_tablas.kpp`, verificado en campo: imprime **72**. Si dos declaraciones del mismo nombre difieren en tamaño, el tamaño compartido es el **máximo** declarado. Ojo con la excepción: un arreglo declarado dentro de un bloque interior con el mismo nombre (sombra) conserva su propio espacio.

Este mecanismo es también el patrón para compartir estado entre funciones en un lenguaje sin variables globales: la tabla compartida hace de memoria común, y las funciones se comunican a través de ella. Así está escrita la primera pieza de Stage 1 (el clasificador del lexer, capítulo 20).

11. Estructuras: struct

Un `struct` agrupa campos con nombre bajo un nuevo tipo:

```
struct Persona
{
    int edad;
    string nombre;
};
```

- La declaración termina con punto y coma después de la llave de cierre (`};`).
- Los campos solo pueden ser de tipo `int` o `string`. No se pueden anidar structs (un campo no puede ser de tipo struct) ni declarar arreglos de structs: son limitaciones conocidas (capítulo 19).
- El compilador rechaza campos duplicados dentro del mismo struct y estructuras con nombres repetidos.

11.1. Instancias y acceso a campos

Cada variable de tipo struct tiene su propio estado, y los campos se leen y escriben con el operador punto:

```
struct Persona
{
    int edad;
    string nombre;
};

int main()
{
    Persona p;
    Persona q;

    p.edad = 25;
    p.nombre = "Ana";

    q.edad = 40;
    q.nombre = "Luis";

    print("PERSONA P");
    print_int(p.edad);     // 25
    print(p.nombre);       // Ana

    print("PERSONA Q");
    print_int(q.edad);     // 40
    print(q.nombre);       // Luis

    p.edad = 30;

    print_int(p.edad);     // 30: cambió p
    print_int(q.edad);     // 40: q intacta

    return 0;
}
```

Es el ejercicio `test_l13_struct_campos.kpp` de la suite: modificar `p` no toca a `q`. Dos structs distintos pueden tener campos con el mismo nombre sin conflicto (ejercicio `test_struct_campos_independientes.kpp`).

12. Enumeraciones: enum

Un `enum` define un conjunto de constantes con nombre:

```
enum Color
{
    ROJO,
    VERDE,
    AZUL
};
```

Puedes asignar valores explícitos; los siguientes continúan la secuencia:

```
enum Estado
{
    Inactivo = 0,
    Activo = 10,
    Pausado    // vale 11
};
```

Y acceder a un valor con su nombre calificado:

```
int main()
{
    return Color::Verde;
}
```

- La declaración termina con `};`.
- El compilador rechaza nombres de enumeradores repetidos y valores inválidos (pruebas negativas de la suite).

13. Entrada y salida de archivos

K++ lee y escribe archivos de KolibriOS con dos funciones integradas. Los datos viajan en arreglos `int`, **un byte por celda**:

```
int n = file_read("DATOS.TXT", datos);          // lee el archivo en el arreglo
int r = file_write("DATOS.TXT", datos, 70000);  // escribe 70000 celdas
```

- `file_write(ruta, arreglo, cantidad)` escribe `cantidad` celdas como bytes. Devuelve **0** si todo fue bien.
- `file_read(ruta, arreglo)` lee el archivo completo en el arreglo y devuelve el **número de bytes leídos**, o un valor negativo si la lectura falló.
- La **ruta es un literal** entre comillas, en mayúsculas estilo 8.3 de KolibriOS (`"BLOQUES.TXT"`). Una ruta en una variable no está admitida.
- Los archivos grandes se mueven en **bloques de 64 KiB** por dentro (sobre la función 70 del sistema): un arreglo de 70 000 celdas se escribe y se lee sin problema.

El ejercicio de referencia es `file_io_bloques.kpp`: llena 70 000 celdas con `i % 251`, las escribe, limpia el arreglo, lo relee y comprueba que la suma coincide. Su salida en KolibriOS es el sello de la pieza:

```
0        // file_write devolvió 0
70000    // bytes leídos
8746781  // suma antes de escribir
8746781  // suma después de leer: idéntica
```

Regla de plataforma (importante): *en KolibriOS, cada archivo se puede leer **una sola vez por proceso**. Si tu programa intenta leer la misma ruta dos veces en la misma ejecución, la segunda lectura devuelve 1 y **no toca el arreglo**. No es un límite del compilador: es el comportamiento verificado del sistema, con todas las sondas. La regla de diseño que se deriva es sencilla: **lee cada archivo una vez por corrida y trabaja con el contenido ya en memoria**. Es exactamente el flujo que usa un compilador, y el que sigue Stage 1.*

14. Comentarios

K++ admite **una sola** forma de comentar: `//` hasta el fin de la línea.

```
int main()
{
    // Esto es un comentario de una línea
    print("Los comentarios no afectan al programa");
    return 0;
}
```

- Los comentarios de bloque `/* ... */` **no existen** en K++. No es un olvido: fue una decisión de diseño (octubre de 2026) — una sola forma de comentar, sin ambigüedad.
- Cuidado: un `/` solitario sigue siendo el operador de división. `//` comenta; `/` divide.

15. Errores comunes y mensajes del compilador

El compilador de K++ detecta tres familias de errores: léxicos, sintácticos y semánticos. Estos son los más frecuentes:

Error	Ejemplo que falla	Mensaje típico
Olvido del punto y coma	int numero = 10	Error sintáctico: se esperaba ;
Variable no declarada	print_int(numero); sin declarar numero	Error semántico: variable no declarada
Tipos incompatibles	int numero = "hola";	Error semántico: tipos incompatibles
Declaración global inválida	código fuera de una función/struct/enum	Se esperaba una declaración global (struct, enum o función)
Índice fuera de límites	valores[9] en un arreglo de 4	Error semántico: índice fuera de límites para el array
Número de argumentos incorrecto	llamar suma(1) a una función de dos parámetros	El compilador lo rechaza en el análisis semántico
Llamada como instrucción suelta	suma(1, 2); en su propia línea	Error sintáctico: las llamadas viven dentro de expresiones (capítulo 9)

Consejo: *lee el mensaje con calma: K++ te dice qué esperaba encontrar y normalmente el problema está en esa línea o en la anterior (casi siempre un ; olvidado).*

16. Compilar y ejecutar tu programa

Paso 1: escribe tu programa. Guarda tu código en un archivo con extensión `.kpp`, por ejemplo `mi_programa.kpp`.

Paso 2: compila.

```
./bin/kpp mi_programa.kpp
```

El compilador ejecuta su pipeline completo: análisis léxico → sintáctico → semántico → generación de IR → backend x86 → emisor → escritor del ejecutable. En pantalla verás la cabecera `K++ COMPILER`, el AST de tu programa, el IR y el volcado del código máquina. Compilar un programa típico toma milisegundos.

Paso 3: obtén el ejecutable. El resultado cae en la carpeta `Output/`: un archivo `.bin` (y su `.raw` gemelo), un ejecutable nativo de KolibriOS — x86 de 32 bits con su cabecera MENUET01 —.

Al copiarlo al disco de KolibriOS, el archivo va sin extensión. Quita el `.bin` y déjalo con su nombre solo y completo: `lexer_clasifica.bin` pasa a la imagen como `LEXER_CLASIFICA`, sin punto ni extensión de ningún tipo. Así funciona KolibriOS: sus programas se reconocen por la cabecera, no por la extensión, y un ejecutable con `.bin` no es la forma en que el sistema los trata. Es la misma regla que el Mini Entorno ASM aplica con sus nombres 8.3.

La suite de pruebas. El repositorio trae `validar_test.sh`, que recompila el compilador y pasa la batería completa de ejercicios positivos y negativos. Una corrida sana cierra con: positivos 60, negativos 11, FAIL 0 sobre los archivos versionados. Es la manera de comprobar que cualquier cambio en el compilador no rompió el lenguaje.

Flujo de trabajo recomendado: *el repositorio incluye `kpp_bootstrap_studio.py` (K++ Bootstrap Studio), un mini-IDE gráfico — editor, compilación y desensamblado del binario — para iterar rápido durante el desarrollo. El Studio encuentra el compilador por sí mismo en cualquier clon del repo: su raíz sale de la ubicación del propio script (y la variable de entorno `KPP_ROOT` puede sobreescribirla). Tiene su propia guía: `Guia_Bootstrap_Studio_KPP.pdf`.*

17. Bajo el capó: la ABI y el ejecutable

Este capítulo es opcional para programar, pero es el corazón del diseño de K++.

17.1. La convención de llamada (ABI)

K++ usa una convención propia, sencilla y completamente regular:

- El **primer** argumento viaja en el registro **EAX**.
- El **segundo** argumento viaja en **EBX**.
- Del **tercero** en adelante, los argumentos viajan **por la pila**: el llamador los empuja en orden inverso (del último al tercero) y, al volver la llamada, limpia la pila sumando `4 × (N − 2)` a ESP, siendo N el total de argumentos.
- El **valor de retorno** vuelve siempre en **EAX**.
- Las funciones se apoyan en el marco clásico de pila (EBP), lo que hace posible la recursión: cada invocación tiene su marco.

Históricamente los argumentos por pila fueron la última pieza de Stage 0 (el «ciclo 4») y su sello es el ejercicio `sonda_ciclo4.kpp`: 15, 15, 40, 40, 40, 40, 40, 40 en KolibriOS.

17.2. CALL y RET: las instrucciones que hacen posibles las funciones

Toda la convención anterior se ejecuta con dos instrucciones x86:

- **`CALL destino`**: empuja en la pila la dirección de la instrucción siguiente (la dirección de retorno) y salta a la función.
- **`RET`**: saca de la pila esa dirección y vuelve exactamente ahí.

Son una pareja inseparable: cada `CALL` necesita su `RET`, y la pila tiene que quedar balanceada — si una función consume más o menos de lo que debe, el retorno salta a cualquier parte y el programa muere. Por eso la batería de llamadas del capítulo 20 se auditó instrucción por instrucción comprobando que los `call`/`ret` estuvieran balanceados.

Así se ve una llamada de K++ en el código máquina real (del desensamblado de `lexer_clasifica`, capítulo 20). El llamador, tras preparar los argumentos, salta con un `CALL` relativo:

```
E8 C2 FB FF FF    call clasifica
```

Y la función llamada construye su marco y lo deshace al salir:

```
55                push ebp          ; guarda el marco del llamador
89 E5             mov ebp, esp      ; marco propio
81 EC 44 00 00 00 sub esp, 0x44     ; espacio para sus variables locales
...                                 ; el cuerpo de la función
89 EC             mov esp, ebp      ; deshace el espacio local
5D                pop ebp           ; restaura el marco del llamador
C3                ret               ; vuelve a la dirección que dejó CALL
```

La recursión (capítulo 9.4) no es magia aparte: es este mismo mecanismo repetido. Cada `CALL` apila un marco nuevo y cada `RET` desapila exactamente uno — la función puede llamarse a sí misma porque cada invocación tiene su marco, sus parámetros y su dirección de retorno propios. Puedes ver estos bytes tú mismo: el Bootstrap Studio desensambla el payload de cada `.bin` que compilas (salta la cabecera de 36 bytes) y los `call` y `ret` aparecen en claro.

17.3. INT 0x40: la puerta al sistema

Un programa de KolibriOS corre en modo protegido, sin acceso directo al hardware ni a la pantalla. Todo lo que necesita del sistema — dibujar la ventana, escribir texto, leer el teclado, tocar un archivo, terminar — lo pide por una única puerta: la interrupción de software **`INT 0x40`** (bytes `CD 40`). El número de función viaja en **EAX** y los argumentos en los demás registros; al ejecutarla, el procesador salta al kernel con privilegio — la instrucción en sí es ordinaria, lo privilegiado es lo que ocurre dentro, en el núcleo — y al volver, el programa sigue en la instrucción siguiente.

En K++ nunca escribes `int 0x40` a mano: **lo hace el runtime** cada vez que usas `print`, `print_int`, `file_read` o `file_write`. Las funciones que tu programa usa sin saberlo, tomadas del runtime

real:

Función (EAX)	Servicio	Quién la usa en tu programa
0	Definir y dibujar la ventana	El runtime, al arrancar
4	Escribir texto en la ventana	print y print_int (al final de su cadena)
2	Leer código de tecla	El bucle de eventos
10	Esperar el siguiente evento	El bucle de eventos
12.1 / 12.2	Abrir / cerrar el redibujo	El runtime, al repintar
17	Leer identificador de botón	El bucle de eventos
70	Leer y escribir archivos	file_read y file_write (capítulo 13)
−1	Terminar el proceso	El runtime, al salir

Los bytes se ven en el desensamblado de cualquier `.bin`. Dos ejemplos de `lexer_clasifica`: escribir texto es `B8 04 00 00 00` (`mov eax, 4`) seguido de `CD 40`; y terminar el proceso es `B8 FF FF FF FF CD 40` — EAX a −1 y la misma puerta.

Este bucle de eventos del runtime explica también el detalle señalado en 17.4: cuando llega un evento de redibujo, el runtime repinta la ventana y vuelve a ejecutar `main`.

17.4. El ejecutable

El backend emite directamente el archivo final: **no hay ensamblador externo ni enlazador** en la tubería. El `.bin` es un binario plano de KolibriOS con esta cabecera de 36 bytes, calculada por el propio compilador:

Desplazamiento	Contenido
0x00	Firma "MENUET01"
0x08	Versión: 1
0x0C	Punto de entrada: 0x24
0x10	Fin de la imagen
0x14	Memoria total = fin de imagen + 0x1000
0x18	Tope de pila = memoria total
0x1C, 0x20	Parámetros y ruta: 0

Detrás de la cabecera va el código x86 y, al final, la cadena de firma del runtime: `K++ - KolibriOS`.

Un detalle observable del runtime: cuando la ventana del programa se redibuja, el runtime vuelve a ejecutar `main`. Es el comportamiento normal de los programas de este entorno, y conviene tenerlo presente al diseñar programas con efectos laterales (por ejemplo, escribir un archivo al arrancar).

18. Filosofía de diseño

K++ se construyó sobre la marcha, resolviendo problemas reales a medida que aparecían. Sin proponérselo, varias de sus decisiones coinciden con las que el lenguaje Go tomó deliberadamente años antes. No es una copia: es convergencia — las mismas presiones (un solo programador, compilación sin fricción, cero maquinaria que no se pague a sí misma) producen las mismas respuestas.

- **Sin archivos de cabecera.** En C, cada archivo se compila por separado y los `.h` existen para declarar lo que vive en otro archivo. El compilador de K++ lee el programa entero y el análisis semántico corre cuando el AST ya está completo: las declaraciones *son* las definiciones, las llamadas hacia adelante funcionan y no hay nada que duplicar ni mantener en paralelo.
- **Sin enlazador.** Un paso menos que la tubería clásica de C: del IR se pasa al binario final.
- **Sin herencia ni clases.** Los structs son datos planos y las funciones trabajan sobre ellos. La herencia resuelve problemas de jerarquías enormes de objetos que estos programas no tienen; lo que no se necesita, no se construye.
- **Sin variables globales.** El estado compartido se hace con arreglos por nombre (capítulo 10.1): explícito y visible.
- **Una sola forma por cosa.** Un solo tipo de comentario (`//`), un solo mecanismo de incremento (`i = i + 1`), una sola convención de llamada.
- **La máquina sin distracciones.** K++ es el lenguaje de una máquina pensada solo para trabajar: KolibriOS como base, binarios pequeños que caben en un disquete de 1,44 MB junto al sistema y al ensamblador — la cadena completa assembler → sistema → lenguaje, sin nada importado.

19. Limitaciones conocidas (lo que K++ aún no hace)

Stage 0 está cerrado y esta tabla resume con honestidad lo que el lenguaje **todavía** no hace:

Área	Estado
Tipos	Solo int y string. Sin void, bool, float, double, char
Structs anidados	✗ Un campo no puede ser de tipo struct
Arreglos de structs	✗ El parser los rechaza
Arreglos como parámetros	✗ No se pasan: se comparten por nombre (capítulo 10.1)
Operadores ++ y --	✗ No existen: se escribe i = i + 1
Comentarios /* */	✗ No existen, por decisión de diseño
Cortocircuito en && / ||	✗ Ambos lados se evalúan siempre
Ruta de archivo en variable	✗ file_read/file_write exigen literal
Segunda lectura del mismo archivo	✗ La plataforma devuelve −1 (capítulo 13)
Arreglos locales por invocación (L17)	Diferido a conciencia: hoy un arreglo es compartido por nombre entre invocaciones recursivas. La pieza L17 lo llevará al marco de cada llamada; su aceptación está escrita: el ejercicio debe imprimir 11, 33, 66, 66
Self-hosting completo	Stage 1 en curso: el compilador aún no se compila a sí mismo; su lexer se construye por rebanadas verificadas

20. Referencia de ejercicios verificados L01–L17

Cada ejercicio de la suite oficial y su estado, tal como lo registra el proyecto. Los capítulos de este manual se construyeron sobre los ejercicios marcados ✔.

ID	Tema	Estado	Archivo de referencia
L01	Tipos int y string	✔ PASS	test_l01_tipos.kpp
L02	Variables e inicializadores	✔ PASS	test_l02_variables.kpp
L03	Operaciones aritméticas (+ − * / %)	✔ PASS	test_l03_aritmetica.kpp
L04	Funciones y retornos	✔ PASS	test_l04_funciones.kpp
L05	Parámetros, argumentos y llamadas	✔ PASS	test_l05_parametros.kpp
L06	Impresión de literales	✔ PASS	test_l06_impresion.kpp
L07	struct: declaración, instancias y campos	✔ PASS	test_l07_struct.kpp, test_l07_struct_funcional.kpp
L08	enum	✔ PASS	test_l08_enum.kpp, test_enum_valores.kpp
L09	Operadores lógicos y relacionales	✔ PASS	kpp-l09-operadores-audit-20261009.kpp
L10	if / else	✔ PASS	test_l10_if_else_audit_20261009.kpp
L11	while	✔ PASS	test_l11_while_audit_20261009.kpp
L12	Ámbitos y shadowing	✔ PASS	test_l12_scope_audit_20261009.kpp
L13	Campos de struct con punto	✔ PASS	test_l13_struct_campos.kpp
L14	Errores sintácticos y semánticos	✔ PASS	batería de negativos L14
L15	Recursión	✔ PASS — sellada en campo	test_l15_recursion.kpp
L16	Llamadas anidadas	✔ PASS	L16_llamadas anidadas.kpp, L16_test_nested_call.kpp
L17	Arreglos en el marco por invocación	Diferida	l17_arreglos_marcos.kpp (aceptación: 11, 33, 66, 66)

Piezas del cierre de Stage 0 (2026-11-08), todas con sello de campo:

Pieza	Ejercicio	Sello en KolibriOS
E/S de archivos en bloques de 64 KiB	file_io_bloques.kpp	0, 70000, 8746781, 8746781
Arreglos alias por nombre	sonda_tablas.kpp	72
Argumentos por la pila (3º en adelante)	sonda_ciclo4.kpp	15, 15, 40, 40, 40, 40, 40, 40
Comentarios //	suite completa	60 positivos / 11 negativos / 0 fallos

Batería de llamadas a funciones (verificada por auditoría de binario). Nueve pruebas dedicadas a llamadas y parámetros, compiladas con `bin/kpp` y verificadas instrucción por instrucción en el desensamblado (`objdump -m i386 -Mintel` sobre el payload del `.bin`): cabecera MENUET01 válida, `call`/`ret` balanceados, argumentos en EAX/EBX según la convención interna y aritmética correcta.

Archivo	Qué comprueba	Resultado esperado	Estado
test_function_call_variables.kpp	Pasar variables como argumentos	Retorno 30	✔ Verificada
test_function_call_expression.kpp	Expresión como argumento (x + 5)	Retorno 35	✔ Verificada
test_function_call_two_expressions.kpp	Expresiones en ambos argumentos	Retorno 55	✔ Verificada
test_function_call_forward.kpp	Llamar función definida después de main	Imprimir 20	✔ Verificada
test_function_call_store.kpp	Guardar el retorno en una variable	Termina con retorno 0	✔ Verificada
test_function_call_print.kpp	Imprimir el retorno de una llamada	Imprimir 10	✔ Verificada
test_function_call_noprint.kpp	Retornar directamente el resultado de una llamada	Retorno 10	✔ Verificada
test_function_parameters_use.kpp	Usar parámetros en una suma	Retorno 30	✔ Verificada
test_una_sola_llamada.kpp	Llamada simple con dos argumentos	Imprimir 30	✔ Verificada

Stage 1, primera rebanada. `bootstrap/stage1/lexer_clasifica.kpp` es el primer ladrillo del lexer escrito en K++: clasifica los caracteres del fragmento `int a=5;` en clases (1 = letra, 2 = dígito, 3 = blanco, 4 = resto) usando dos arreglos compartidos por nombre entre `main` y la función `clasifica`. Compilada con Stage 0; su salida esperada al correr en KolibriOS es **1 1 1 3 1 4 2 4**.

Manual generado a partir del código fuente real del proyecto K++ (repositorio github.com/saykol31/KPP, master 06e94e3), verificando cada ejemplo con el compilador `bin/kpp`, la suite oficial (60 positivos / 11 negativos / 0 fallos) y las pruebas de campo en KolibriOS que sellaron Stage 0 el 8 de noviembre de 2026. K++ es software experimental: la sintaxis puede cambiar entre versiones.